Software Architecture and Design Document

For Advising Assistant

Author(s): Oluwajomiloju Adejumo, Keishon Boose, Rabindra Giri, and Evans Smith.
University of Southern Mississippi, Front End – Advising Assistant Group 3

Purpose

This document serves as the specification for the architecture and component design for the Advising Assistant software system. It serves as a blueprint for the development of the software system.

Scope

The advising assistant app should be an app that can assist students with making and planning schedules for their future semester. Some other functions will be telling when the person can or cannot graduate. Also, they should be able to help students plan their schedule for the upcoming semester. The app should be able to store a curriculum that a student is enrolled in and compare the classes that have already been taken to the ones that need to be taken. And it should generate a list of classes that the student should take. This app targets college students and/or parents of future college students.

Overview

The advising assistant uses a client-server architecture to facilitate the user to navigate through their courses and allow them to perform different functions. The front end of the advising assistant will provide user with abilities to add the courses in the web application and help in tracking the courses with a checklist of what has been completed and what is remaining to calculate and show in the front end to show how much time is remaining for the completion of the degree and their track of when they are going to finish in an interactive and intuitive way. We will add an interactive user interface for students to input their academic history and add the most common courses available at the university as recommendations and visualize the progress towards graduation. The server-side architecture will handle the connection to database which will typically be done by a backend tech stack with Node and Express.js. The database will be connected through a connection pool depending on the setup on the backend. As our team is designing the front end and backend, we will be using React.js for the front end and Tailwind CSS for design. We will also be using Git as a version control system for our frontend and backend. The primary way of sending user data will be with controls like form fields and buttons triggering RESTful APIs with axios package on the front end to the application backend with node.js, and the data will be processed in the backend and recommendation will be given for courses on basis of that with RESTful APIs.

Overall Description

The primary goals and objective of the Advising Assistant is to help students plan their semester efficiently to ensure that students stay on track of graduation while also recommending electives based on the students' interest. The system should also effectively show the accurate description of courses and helpful metrics like how many courses are left and the estimated graduation date so that advising can be simpler and more intuitive while replacing the traditional ways of complex advising and taking courses.

The users being targeted by Advising Assistant are undergraduate students who must face the complex process of advising. We realized that students must have access to a user friendly and intuitive way of tracking courses and receiving appropriate feedback on the progress of their degree. So, Advising Assistant targets these university students and brings a new experience to advising. We also understand the limitations of a computer program and have advisor's account to answer questions asked by students with a specific major.

Our target operating system will be desktop and if we are able to make the website responsible in the limited time constraints then mobile as well. We are creating a web app so the software required to interact with the advising assistant will be a browser. The app will require an internet connection to fetch the data of courses on the database from the backend to display to the users.

Components

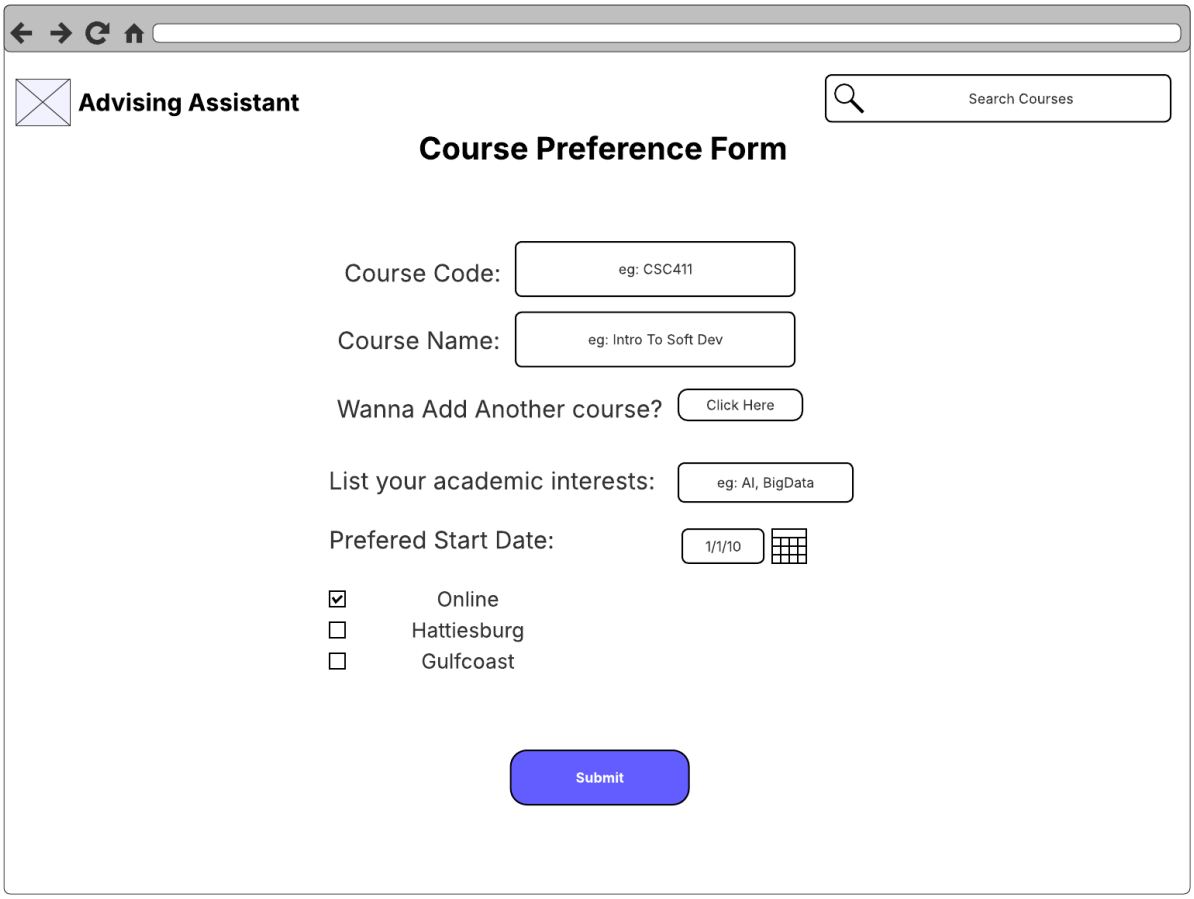
C1: Course Form Component

Description: The form component will provide an input mechanism for the students to input their courses, interests and own preferences which will be used in the backend to recommend courses and establish project dates.

Functional Requirements: Form Validation to Prevent Invalid Entries, Connected with Backend for Checking Duplications, Input Fields to Enter Course Information

Non-Functional Requirements: User friendly, User should be able to navigate without assistance, Accessible, Easy to understand feedback if user inputs invalid entries

Component Design:



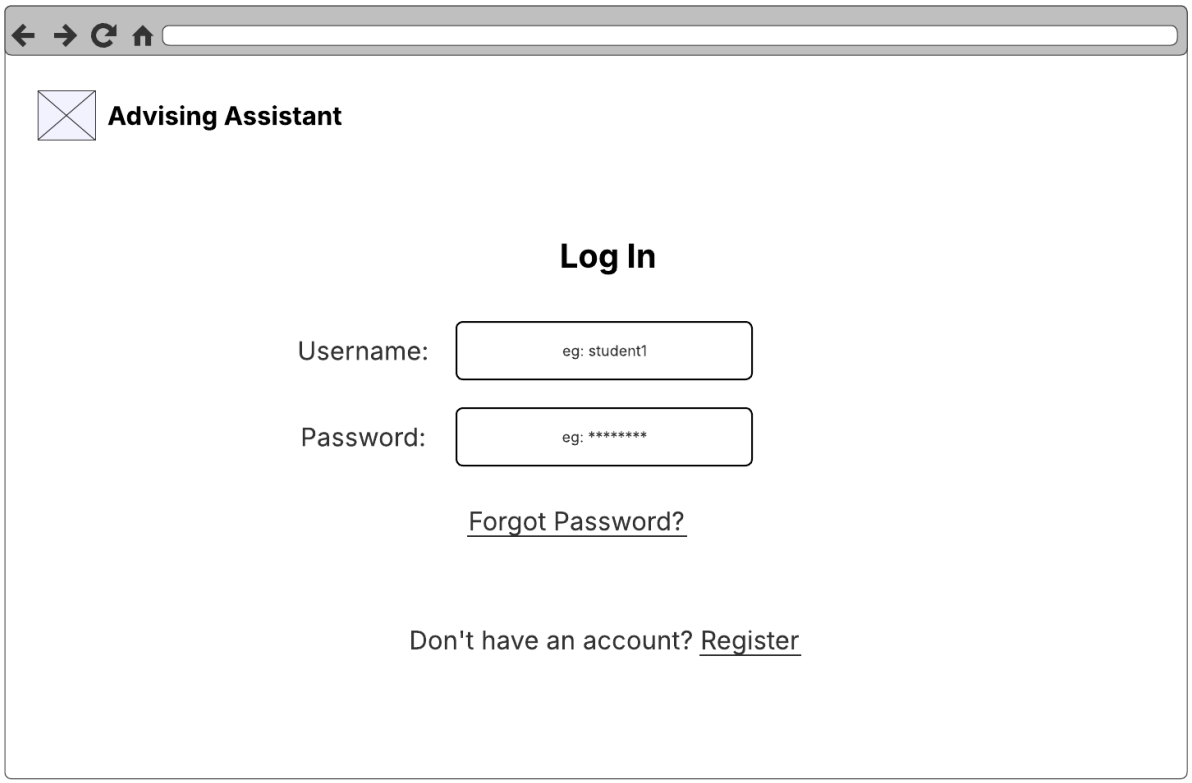
C2: Authentication Form Component

Description: This component handles the authentication component when user registers a new account or logs in to an existing account. This component needs to be connected to backend Authentication component that checks for existing users in the database.

Functional Requirements: Input field for entering registering an account, username and password fields for login account. Data validation and password safety compliance

Non-Functional Requirements: User friendly and should be easy to navigate, visually appealing design and interactive feedback.

Component Design:



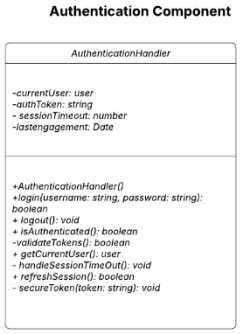
C3: Backend Auth Component

Description: This component is situated in the backend and handles the logic for authentication and includes classes to for the validation of data, check if the user exists in database and if exists raise an error to the frontend and if doesn't exist then the logic for creating the user according to the register and login page logics. Should also use Bcrypt Like Tokenizer for Hashing the Passwords.

Functional Requirements: Include Register User and Login User Classes to Handle Registration Logic and Authentication Logic, Bcrypt Tokens for Hashing Password, JSON Web Tokens for Frontend and Backend Session Management

Non-Functional Requirements: Should be optimized for user so that processing happens under 2 seconds, Secure and Intuitive to Access API Endpoints

Component Design:



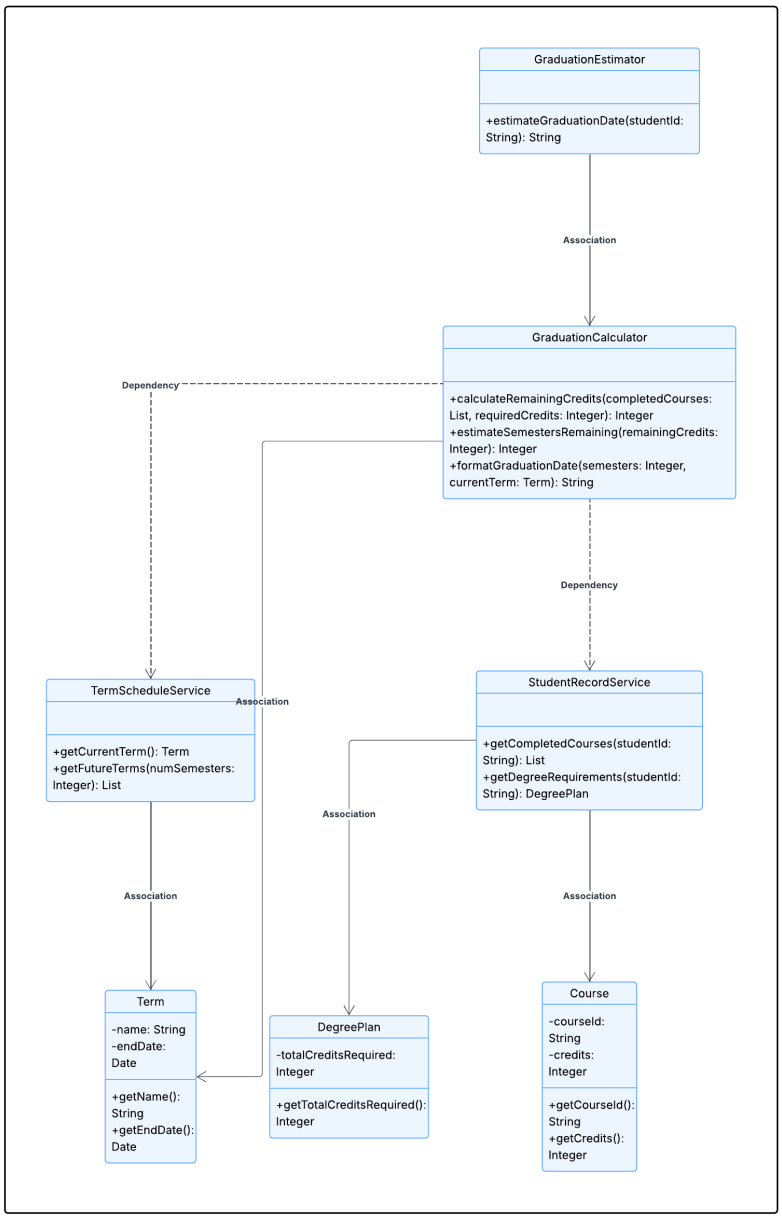
C4: Graduation Estimation Logic Component

Description: This component should analyze the student completed courses and the course that needs to be completed to estimate an accurate graduation date.

Functional Requirements: Included Calculation Logic for the Estimation

Non-Functional Requirements: Should Display Date in Easy-to-Understand Formatting, Adapt dynamically when courses are added/removed

Component Design:



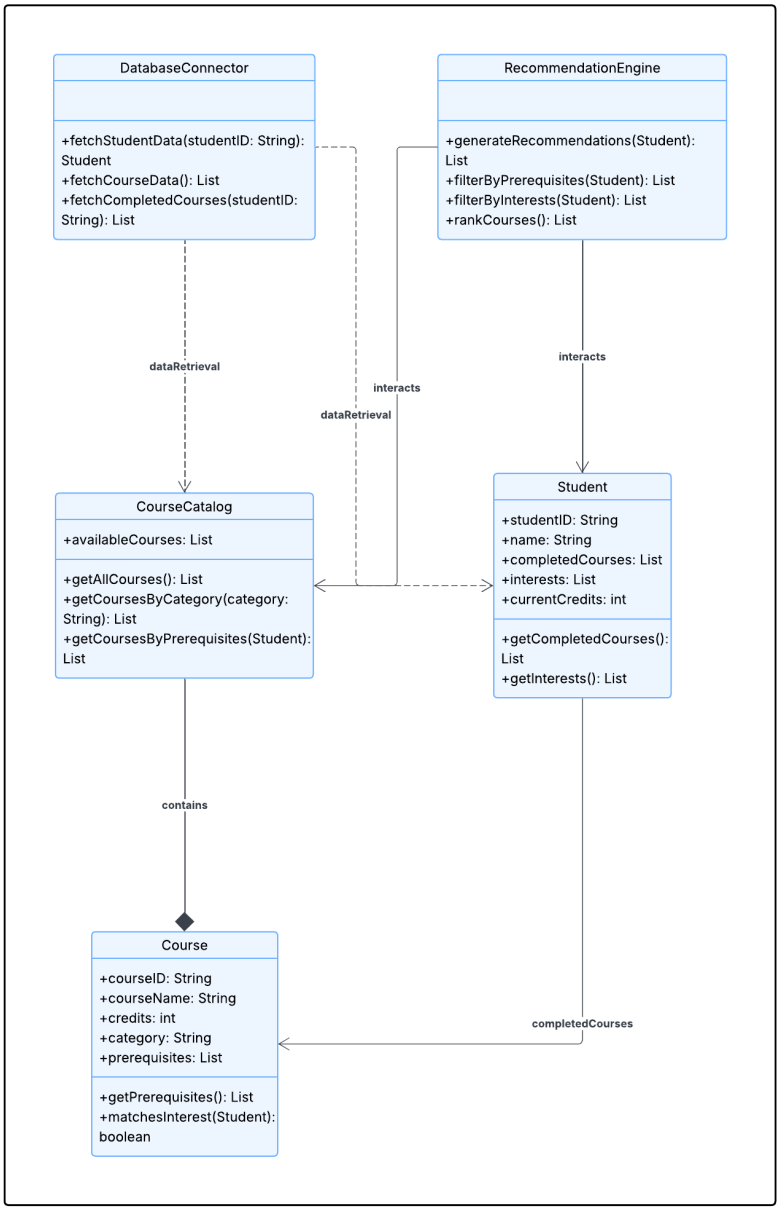
C5: Course Recommendation Component

Description: This component is responsible for generating a personalized course recommendation for the students based on their prerequisite of courses and interest provided from the categories of the courses.

Functional Requirements: Perform Database Checks and Analyze the Student Course Prerequisites and History, Algorithm to Filter Courses with Courses.

Non-Functional Requirements: The course recommendation should be generated under 3 seconds and the changes made by the student should immediately generate another recommendation. The user should have a smooth experience with the process.

Component Design:



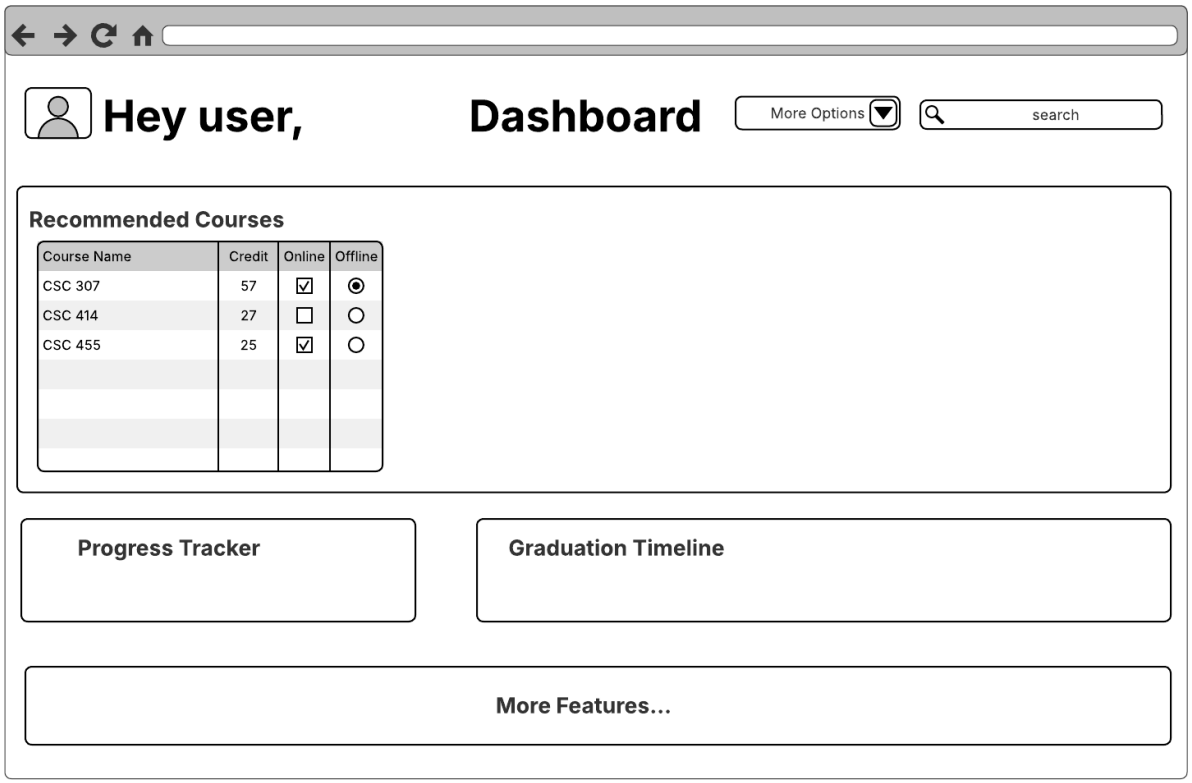
C6: Dashboard User Component

Description: This component should show the first page that the user sees after they login. It should show the overview of the student's academic progress with all the essential features like graduation estimation, the completion progress and their name and log out features.

Functional Requirements: A Display Component That Includes Recommended Courses, Graduation Date Estimation, Frontend Implementation of Backend Logics

Non-Functional Requirements: Load the dashboard in under 3 seconds. Users should be able to navigate all the features without any assistance. Responsive design so that information is not distorted.

Component Design:



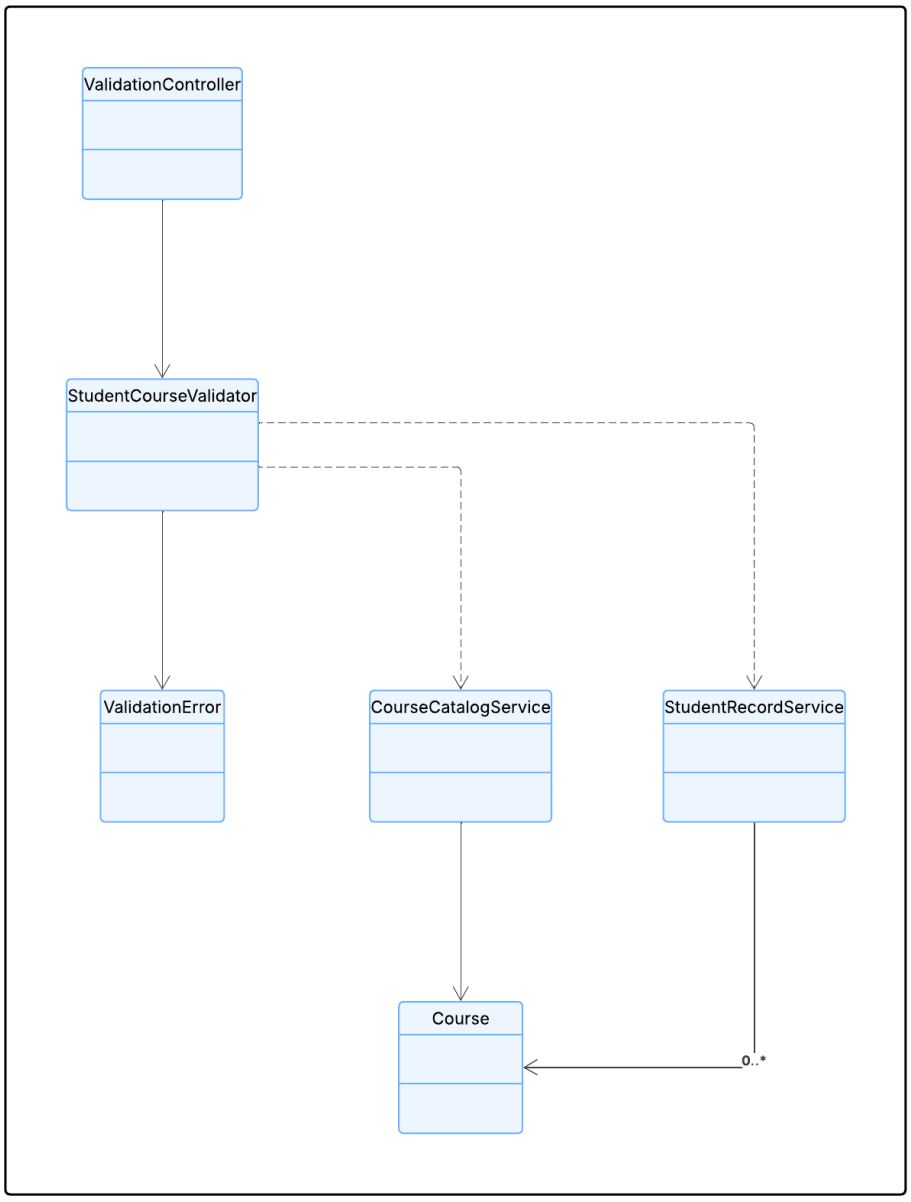
C7: Data Validation Component

Description: This component makes the separation of concern for validation as validation is required in most of the apps for user data. This component is connected to frontend and written in backend and raises errors when there are conflicts. This component has a structure that validates the student course records and related info.

Functional Requirements: Validation Logic for Prerequisite, Course Duplication, Credit Hour Limits

Non-Functional Requirements: Provide responsive errors and feedback in an intuitive manner, should provide feedback under 2 seconds.

Component Design:



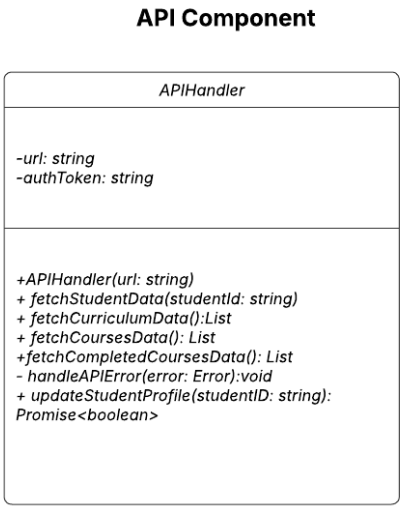
C8: RESTful APIs Endpoint Component

Description: This component ensures that the backend and frontend logic are properly connected. In this component, there is logic inbuilt, and tests written to verify that each API endpoint working as expected and expected data type and requests are sent.

Functional Requirements: Handle error conditions and testing for the API endpoints, Send API requests to endpoints, Handle connection frontend, backend

Non-Functional Requirements: Responsive error handling, properly named API endpoints.

Component Design:



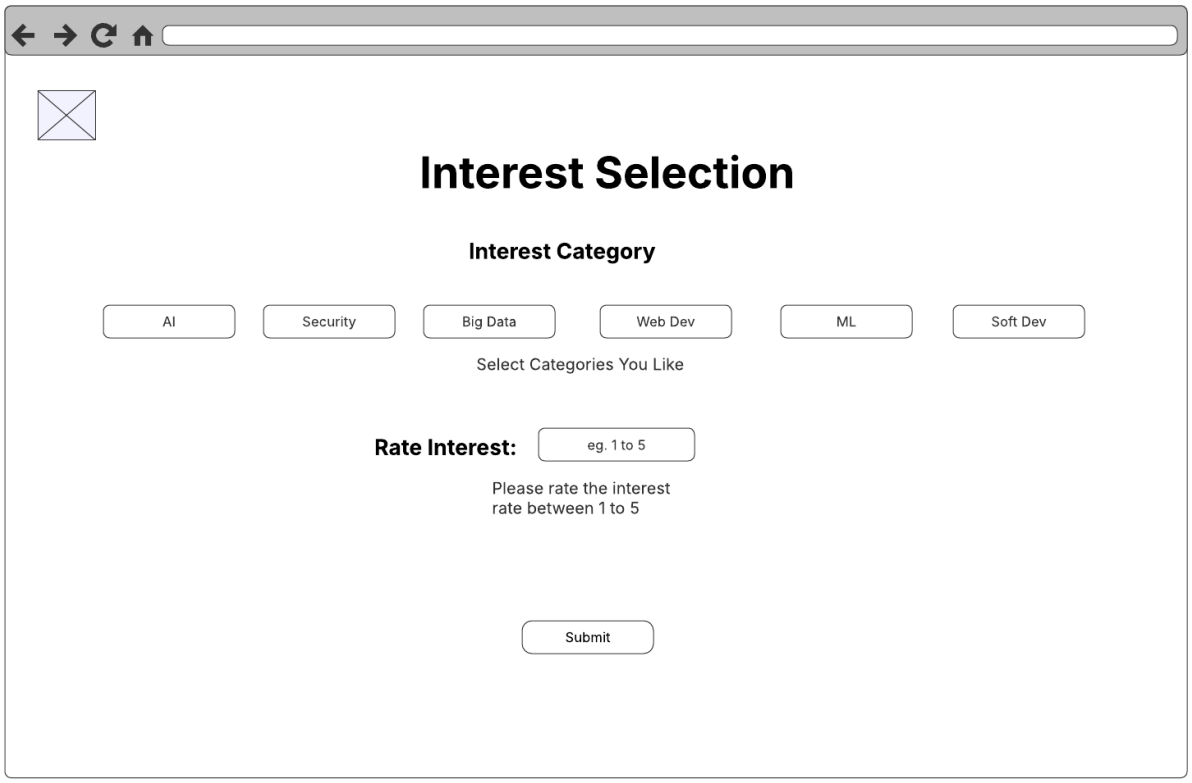
C9: Interest Choice Component

Description: This component is responsible for showcasing different categories like AI, Bigdata and sending that data to display personalized recommendations.

Functional Requirements: Present Selection Screen and Options. Present Choice of 1 to 5 rating of Interested Scale

Non-Functional Requirements: Interactive And Intuitive Design for Categories Selection.

Component Design:



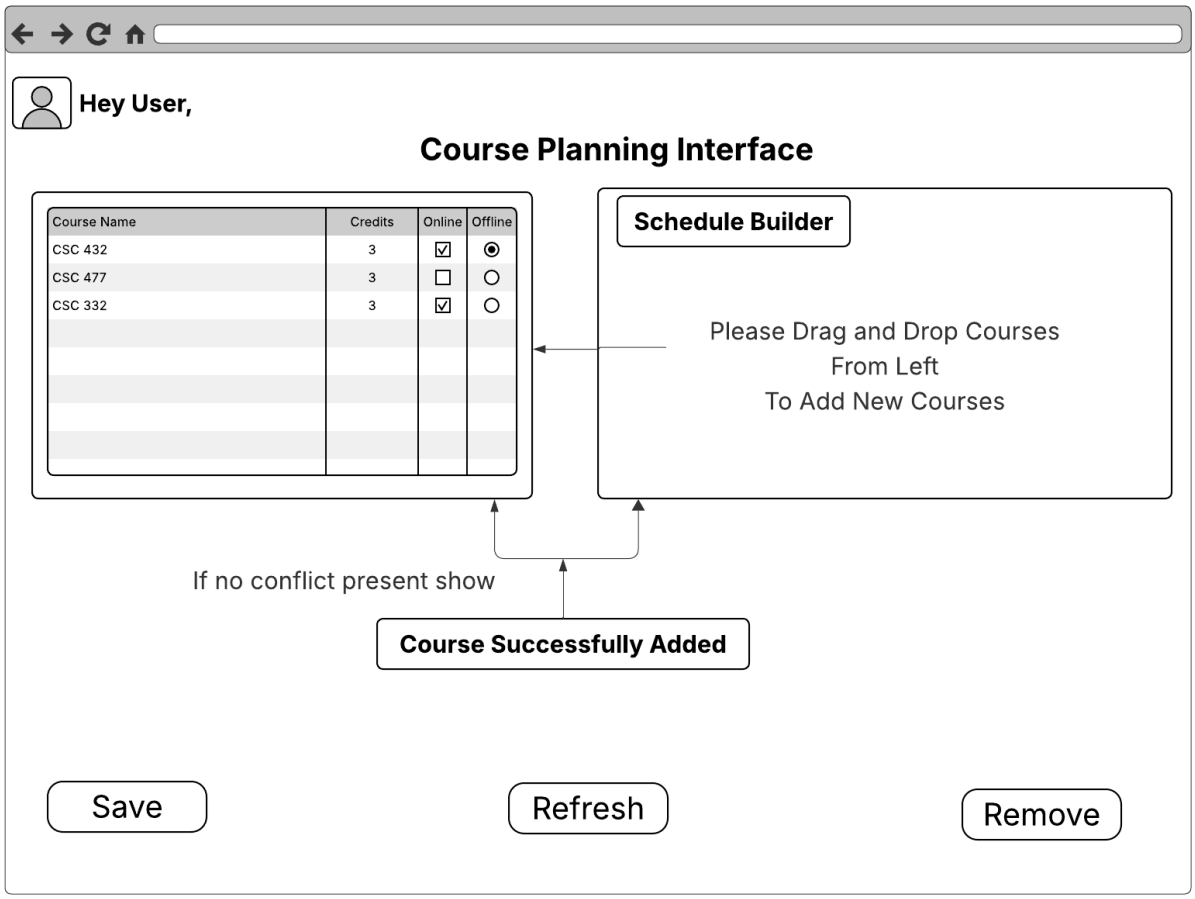
C10: Course Planner Component

Description: This component displays an interface that allows students to select and organize the courses for the upcoming semester. This is an interface to put courses on a schedule builder type of interface to manage the courses and hold them.

Functional Requirements: Display and Search Filter for Available Courses. Option To Remove Courses. Option To Save Newly Added Courses. Validate Schedule Conflicts

Non-Functional Requirements: Interactive and Immediate Response for Conflicts, When A course is added it should be displayed under 2 seconds.

Component Design:



System Architecture

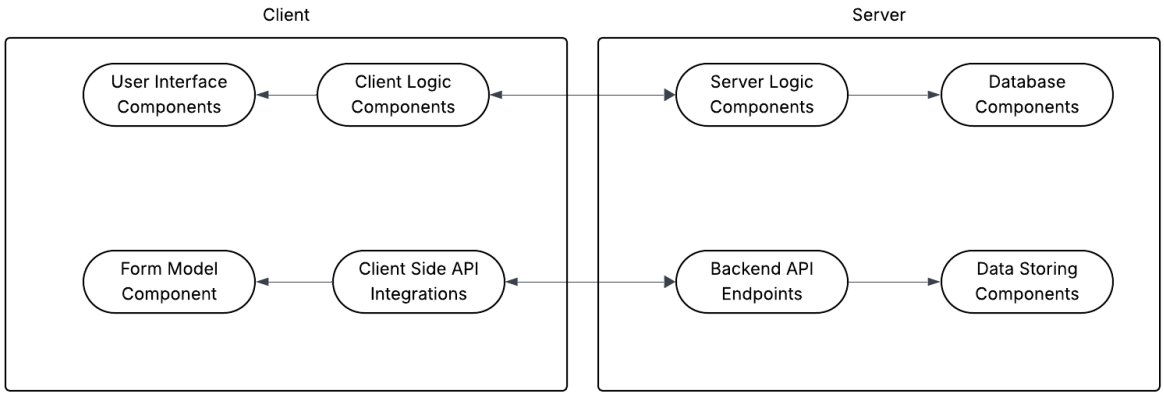
A1: Front End and Back End Interaction

Description: This diagram shows the overall architecture of the system and how backend and frontend are differentiated and situated in each of their side.

Functional Requirements: Frontend built in React.js, Backend built in Node.js, Database Connected Locally or Remotely PostgreSQL and Built API Endpoints For connection.

Non-Functional Requirements: Interactive Design, Security and Scalability.

Model Type: Client/Server Model



A2: Advising Assistant Client Architecture

Description: This diagram shows how the frontend has separation of concerns and modular with the pages.

Functional Requirements: Client Functions Separation of Logic and Functions

Non-Functional Requirements: Modular Design and Separation of Concern

Model Type: Model-View-Controller Architecture

Client Architecture